

Code Explanation

Delta Live Tables Test Code Explanation

This code is designed to test the functionality of Delta Live Tables (DLT) in a Databricks environment. The tests cover the import of the `delta\_live\_tables` module and the correctness of the `customer`, `order`, and other functions within this module.

Overview of the Code

The provided Python code is a test suite for Delta Live Tables. It includes fixtures for creating a SparkSession and mocking the `dlt` module, as well as test functions to validate the behavior of the `customer`, `order`, and other functions.

Step 1: Importing Necessary Modules and Creating Mock DLT Module

The code begins by importing necessary modules, including `pytest`, `os`, `patch`, and `MagicMock` from `unittest.mock`, as well as `SparkSession` and various data types from `pyspark.sql`. It then defines a `MockDLT` class to mimic the behavior of the `dlt` module, which is specific to Databricks.

```
import pytest
import os
from unittest.mock import patch, MagicMock
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType, DateType
import datetime

class MockDLT:
    @staticmethod
    def table(*args, **kwargs):
        def decorator(f):
            return f
        return decorator

    @staticmethod
    def read(table_name):
        if table_name == "customer":
            return mock_customer_df
        elif table_name == "order":
            return mock_order_df
        elif table_name == "ordersummary":
            return mock_ordersummary_df
        return None
```

Step 2: Creating Mock DataFrames for Testing

The test code creates mock DataFrames (`mock\_customer\_df`, `mock\_order\_df`, `mock\_ordersummary\_df`) that will be used to test the functions in `delta\_live\_tables`. These DataFrames are created using the `SparkSession` fixture.

```
@pytest.fixture
def setup_mock_dataframes(spark):
```

```

global mock_customer_df, mock_order_df, mock_ordersummary_df

customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

customer_data = [
    ("C001", "John Doe", "john@example.com", "North"),
    ("C002", "Jane Smith", "jane@example.com", "South"),
    ("C003", "Bob Johnson", "bob@example.com", "East"),
    ("C004", "Alice Brown", "alice@example.com", "West")
]

mock_customer_df = spark.createDataFrame(customer_data, customer_schema)

# ... similar creation for mock_order_df and mock_ordersummary_df

```

Step 3: Patching the DLT Module and Creating a SparkSession Fixture

The `mock\_dlt\_module` fixture patches the `dlt` module with the `MockDLT` class, and the `spark` fixture creates a `SparkSession` for testing.

```

@pytest.fixture(autouse=True)
def mock_dlt_module():
    with patch.dict('sys.modules', {'dlt': mock_dlt}):
        yield

@pytest.fixture
def spark():
    return SparkSession.builder
        .appName("TestDeltaLiveTables")
        .master("local[1]")
        .getOrCreate()

```

Step 4: Testing the Import of Delta Live Tables Module

The `test\_import\_delta\_live\_tables` function tests that the `delta\_live\_tables` module can be imported successfully.

```

def test_import_delta_live_tables(setup_mock_dataframes):
    from src.delta_live_tables import customer, order, ordersummary,
    customeraggregatespend
    assert True

```

Step 5: Testing the Customer Function

The ``test_customer_function`` tests the ``customer`` function by mocking ``spark.read`` to return ``mock_customer_df`` and verifying that the returned DataFrame has the expected columns.

```
def test_customer_function(setup_mock_dataframes, spark, monkeypatch):
    mock_read = MagicMock()
    mock_read.format.return_value.option.return_value.option.return_value.load.return_value = mock_customer_df
    monkeypatch.setattr(spark, 'read', mock_read)

    from src.delta_live_tables import customer
    result_df = customer()

    expected_columns = ["CustId", "Name", "EmailId", "Region"]
    assert all(col in result_df.columns for col in expected_columns)
```

Step 6: Testing the Order Function

The ``test_order_function`` tests the ``order`` function by creating a raw order DataFrame, mocking ``spark.read`` to return this DataFrame, and verifying that the ``TotalAmount`` column is correctly calculated.

```
def test_order_function(setup_mock_dataframes, spark, monkeypatch):
    order_data_raw = [
        ("0001", "Product A", 10.0, 2, "2023-01-15", "C001"),
        ("0002", "Product B", 15.0, 1, "2023-01-20", "C002"),
    ]

    # ... create raw_order_df

    mock_read = MagicMock()
    mock_read.format.return_value.option.return_value.option.return_value.load.return_value = order_data_raw
    monkeypatch.setattr(spark, 'read', mock_read)

    from src.delta_live_tables import order
    result_df = order()

    assert "TotalAmount" in result_df.columns
    sample_row = result_df.filter("OrderId = '0001'").first()
    assert sample_row["TotalAmount"] == sample_row["PricePerUnit"] * sample_row["Qty"]
```